

Table of Contents

1.0 Scope of Work	1
1.1 Introduction	1
1.2 Requirements Analysis	1-2
2.0 ER Model	3
2.1 Detailed Business Rules and Assumptions	3
2.1.1 Customer & Booking	3
2.1.2 Tour Package & Itinerary	3
2.1.3 Tour Guide	3
2.1.4 Booking & Reservation	3
2.1.5 Payment	3
2.1.6 Ticket	4
2.1.7 Flight	4
2.1.8 Hotel	4
2.1.9 Transport	4
2.1.10 Supplier	5
3.0 Redesign and EER	6
3.1 Entity Specialization	6
3.1.1 Service Supertype	6
3.1.2 Reservation Supertype	7
3.1.3 Reservation Subclasses	7
3.1.4 Supplier Subclasses	7-8
3.2 Associative Entities	8
3.2.1 Package Service	8
3.3 Relationship Redesign	8

3.3.1 Reservation & Service	8
3.3.2 Service & Package Service	8
3.3.3 Tour Package & Package Service	8
3.3.4 Supplier & Service	9
3.3.5 Flight Reservation & Ticket	9
4.0 Data Dictionary	10
4.1 Customer	10
4.2 Tour_Package	10
4.3 Booking	10
4.4 Itinerary	10
4.5 Tour_Guide	11
4.6 Package_Guide	11
4.7 Ticket	11
4.8 Payment	11
4.9 Supplier	12
4.10 Airline	12
4.11 Hotel_Provider	12
4.12 Transport_Provider	12
4.13 Service	12
4.14 Flight	13
4.15 Hotel	13
4.16 Transport	13
4.17 Reservation	13
4.18 Flight_Reservation	14
4.19 Hotel_Reservation	14

4.20 Transport_Reservation	14
4.21 Package_Service	14
5.0 Tables and Records	15
5.1 Table Customer	15
5.2 Table Tour_Package	15
5.3 Table Booking	15
5.4 Table Itinerary	16
5.5 Table Tour_Guide	16
5.6 Table Package_Guide	16
5.7 Table Ticket	17
5.8 Table Payment	17
5.9 Table Supplier	17-18
5.10 Table Airline	18
5.11 Table Hotel_Provider	19
5.12 Table Transport_Provider	19
5.13 Table Service	19-20
5.14 Table Flight	20
5.15 Table Hotel	20
5.16 Table Transport	21
5.17 Table Reservation	21
5.18 Table Flight_Reservation	22
5.19 Table Hotel_Reservation	22
5.20 Table Transport_Reservation	22
5.21 Table Package_Service	23
6.0 Script	24-45

7.1 SQL Queries	46-47
7.2 PL/SQL Stored Procedures	47-49
7.3 PL/SQL Functions	49-51

1.0 Scope of Work

1.1 Introduction

The Booking and Reservation System is developed for West Alliance's tourism division, DIY Holiday. It uses to manage and streamline travel-related services. As the company expands its operations in tourism, an efficient computerized system is required to handle customer bookings, tour package selections, reservations and payments in an organized and centralized manner.

This system aims to reduce manual processing, minimize errors and improve service efficiency by integrating all booking-related activities into a single platform. It also ensures proper coordination between customers, suppliers and internal management.

1.2 Requirements Analysis

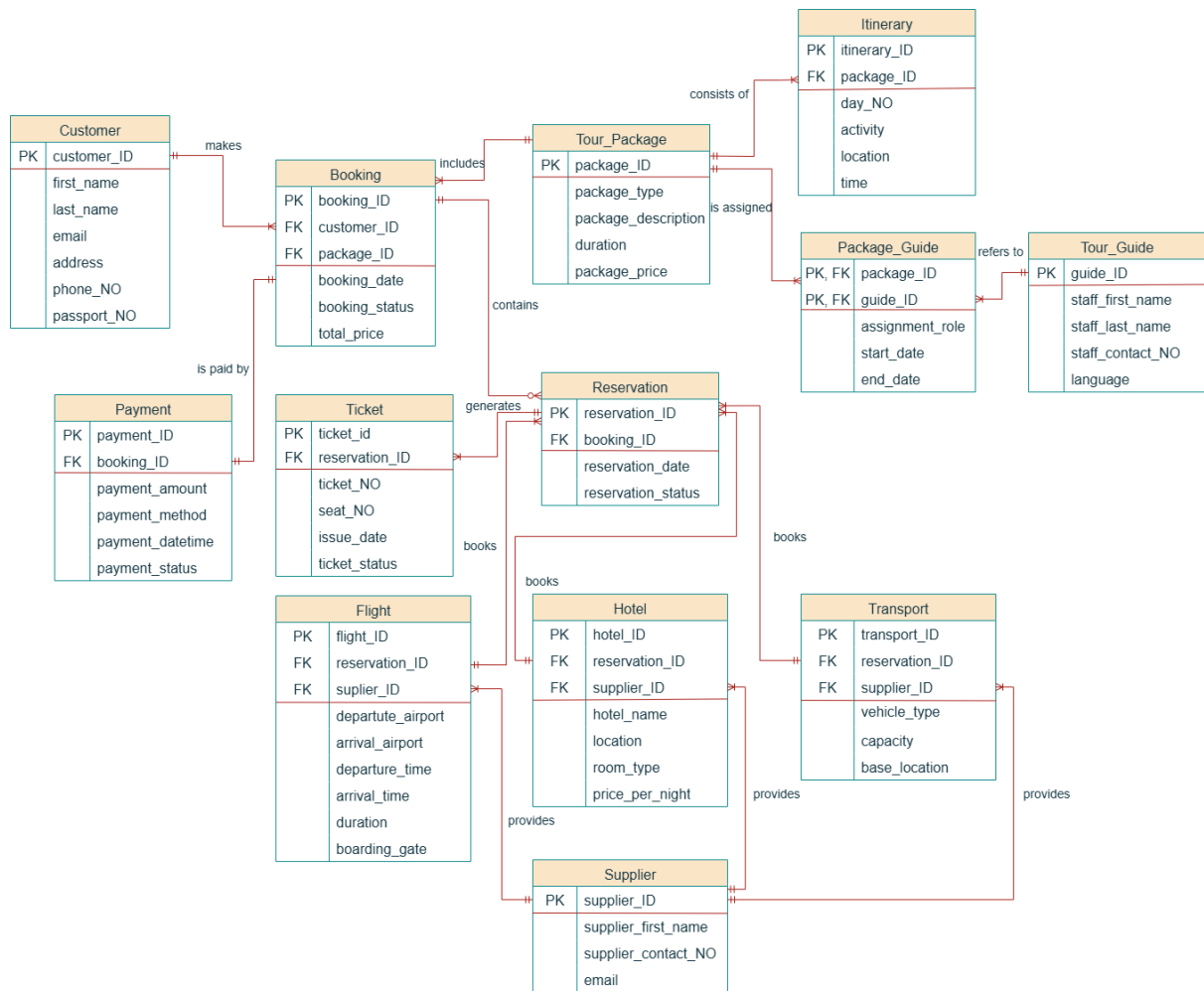
This system will be used to handle all booking and reservation processes associated with the tourism enterprise. Customers will have the opportunity to search for various tour packages and make their reservations through the system. These requirements define how the system manages relationships between customers, bookings, reservations, and services, ensuring that all operations follow structured business rules and maintain data consistency.

The main requirements of the system include:

- *Customer Management:*
The system keeps and manages customer details such as name, contact information and passport number. Only registered customers are allowed to make bookings.
- *Tour Package Management:*
Tour packages available are chosen by customers depending on their itinerary information, price and duration.
- *Booking Management:*
A booking acts as the main transaction record that links customers to selected tour packages and services.
- *Reservation Management:*
For each booking made there will be several reservations that include flights, hotels, transport and other tour package services, all of them having different status information and service information.
- *Payment Management:*
The system handles payment records for each booking, including payment method, date and total amount. A booking is confirmed only after successful payment.
- *Supplier Management:*
The system maintains records of suppliers (airlines, hotels, transport providers) and links them to the services they provide.

- *Status Tracking:*
The system tracks booking and reservation statuses such as pending, confirmed or cancelled.

2.0 ER model



2.1 Detailed Business Rules and Assumptions

2.1.1 Customer & Booking

Business Rules:

- A customer can make multiple bookings.
- Each booking is made by one customer.
- A booking must be associated with exactly one Tour Package.
- A customer is required to provide valid personal details including first name, last name, email, phone number, address, and passport number

Assumptions:

- A customer cannot book multiple packages in a single booking.
- Customer email is assumed to be unique.

2.1.2 Tour Package & Itinerary

Business Rules:

- A tour package is uniquely identified by a package ID and contains attributes such as package type, description, duration, and price.
- Each tour package must have at least one itinerary.

- An itinerary belongs to only one tour package.
- Each itinerary records the day, activity, location, and time, allowing a package to have multiple scheduled activities.

Assumptions:

- day_NO is sequential starting from 1 up to duration.
- Duration of package = number of itinerary days.
- Activities may repeat across different packages.

2.1.3 Tour Guide

Business Rules:

- A tour guide is identified by a guide ID and includes staff name, contact and language.
- A tour guide can be assigned to multiple tour packages and each tour package can have multiple tour guides, forming a many-to-many relationship through the package guide entity.
- Each assignment specifies the role of the guide as well as the start date and end date and a tour guide must be available during the assigned period.

Assumptions:

- A guide can have different roles
- A guide cannot have overlapping assignments for the same package.
- Language indicates languages spoken by the guide.

2.1.4 Booking & Reservation

Business Rules:

- A booking can generate one or more reservations
- Each reservation belongs to one booking.
- Reservations track service arrangements such as flights, hotels, and transport.

Assumptions:

- A booking may exist without reservations initially.
- Each service is tied to a reservation, not directly to booking.

2.1.5 Payment

Business Rules:

- A booking can have multiple payments, but at least one payment is required for confirmation.
- Each payment records the payment amount, payment method, payment date.

Assumptions:

- Partial payments are not allowed.
- Payment status includes values like pending, completed, failed, refunded.
- Total booking price = sum of successful payments.

2.1.6 Ticket

Business Rules:

- A ticket is generated based on a confirmed reservation after successful payment.
- Each reservation can generate one or more tickets depending on the number of services booked.
- Each ticket is associated with exactly one reservation.
Each ticket must have a unique ticket ID and ticket number.

Assumptions:

- A reservation may exist without a ticket until payment is completed and confirmed.
- Each ticket is uniquely identified by a ticket ID.

2.1.7 Flight

Business Rules:

- Each Flight belongs to one reservation and is provided by one supplier.
- Flight must store departure/arrival airports and times.

Assumptions:

- A reservation has at most one flight record.
- Boarding gate duplication in attributes is assumed to be a typo.

2.1.8 Hotel

Business Rules:

- Each hotel belongs to one reservation and is provided by one supplier.
- Hotel must include room type and price per night,

Assumptions:

- One reservation corresponds to one hotel stay.
- Duration of stay is derived from itinerary or package.

2.1.9 Transport

Business Rules:

- Each transport belongs to one reservation and is provided by one supplier.
- Transport should include capacity and base location.

Assumptions:

- Transport is used for local travel within the package.
- One transport per reservation.

2.1.10 Supplier

Business Rules:

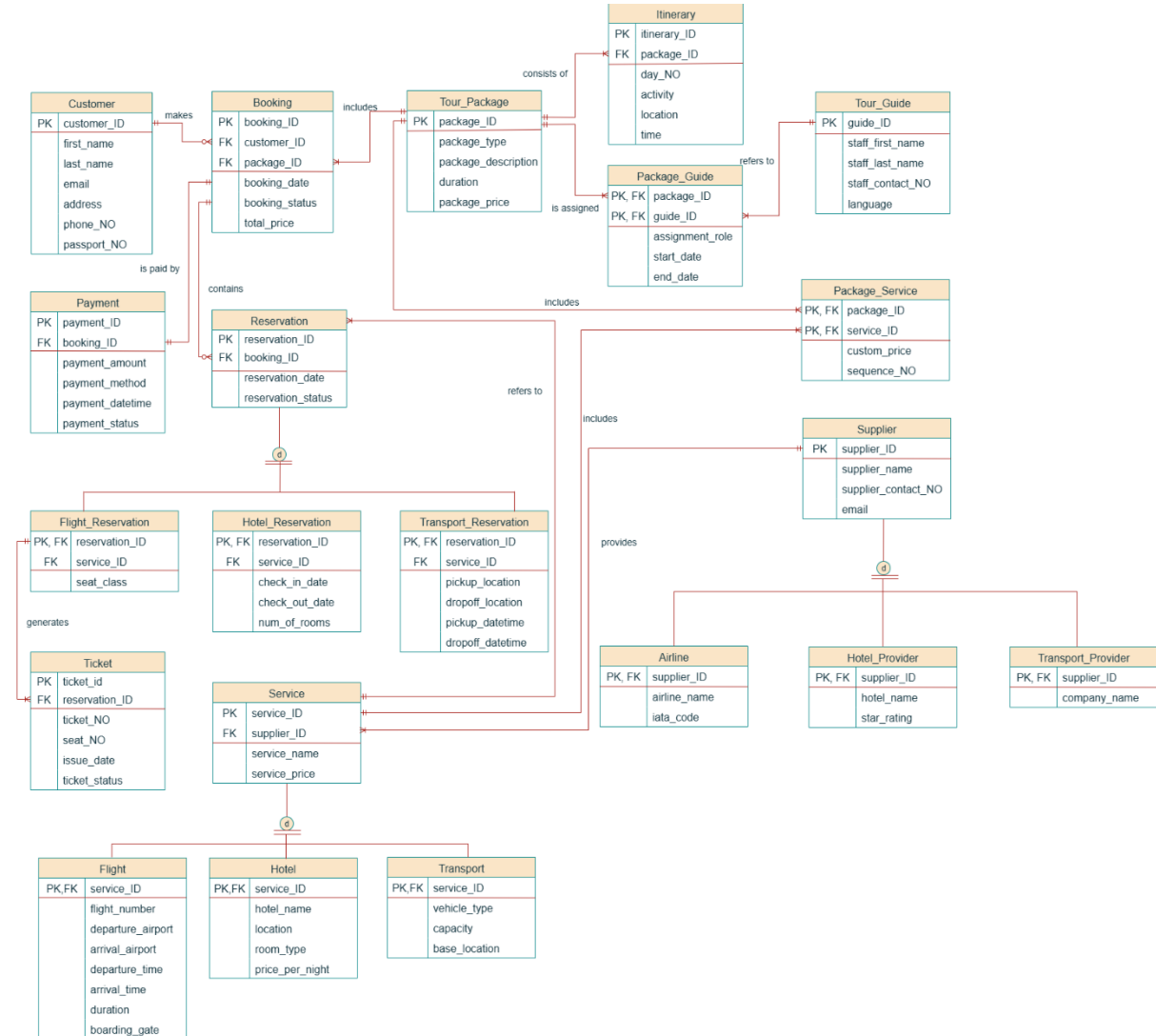
- Suppliers provide services such as flights, hotels, and transport.
- Each service must be associated with a supplier.

Assumptions:

- A supplier may specialize in exactly one service category (airline, hotel provider or transport provider).
- Supplier email is unique.

3.0 Redesign and EER

The system is redesigned into an EER model to support specialization, generalization and complex relationships such as many-to-many associations. The model introduces supertype and subtype entities to reduce redundancy and improve data consistency.



3.1 Entity Specialization

3.1.1 Service Supertype

Business Rules:

- A service is uniquely identified by a service ID.
- A service represents a general category for flight, hotel and transport services.
- Each service must belong to exactly one subtype which is either flight, hotel, or transport.

Assumptions:

- Specialization is total and disjoint.
- Service type is mandatory and cannot be null.
- A service cannot exist without a subtype.

3.1.2 Reservation Supertype

Business Rules:

- A reservation is uniquely identified by a reservation ID.
- A reservation must be linked to one booking.
- A reservation must refer to exactly one service.

Assumptions:

- Specialization is total and disjoint.
- Each reservation handles only one service type.

3.1.3 Reservation Subclasses

i) Flight Reservation

Business Rules:

- A flight reservation is associated with one reservation record.
- It stores flight-specific booking details such as seat class and preferences.

Assumptions:

- Each reservation can have only one flight reservation.
- Flight details are derived from the associated service and are not duplicated.

ii) Hotel Reservation

Business Rules:

- A hotel reservation is associated with one reservation record.
- It stores hotel-specific details such as check-in date, check-out date and room type.

Assumptions:

- Each reservation corresponds to one hotel stay.
- Duration of stay may be derived from the tour package or itinerary.

iii) Transport Reservation

Business Rules:

- A transport reservation is associated with one reservation record.
- It stores transport-specific details such as pickup and drop-off information.

Assumptions:

- Each reservation can have only one transport arrangement.
- Transport is used within the scope of the tour package.

3.1.4 Supplier Subclasses

i) Airline

Business Rules:

- Airline is a subtype of supplier that provides flight services.
- Each airline is uniquely identified through the supplier entity.

Assumptions:

- Specialization is disjoint.
- An airline can provide multiple flight services.

ii) Hotel Provider

Business Rules:

- Hotel provider is a subtype of supplier that provides hotel services.
- Each hotel provider is uniquely identified through the supplier entity.

Assumptions:

- Specialization is disjoint.
- A hotel provider can manage multiple hotel services.

iii) Transport Provider

Business Rules:

- Transport provider is a subtype of supplier that provides transport services.
- Each transport provider is uniquely identified through the supplier entity.

Assumptions:

- Specialization is disjoint.
- A transport provider can provide multiple transport services.

3.2 Associative Entities

3.2.1 Package Service

Business Rules:

- Package service links services to tour packages.
- Each record associates one service with one tour package.

Assumptions:

- A service can be included in multiple tour packages.
- A tour package can consist of multiple services.
- Package service acts as a bridge entity to resolve the many-to-many relationship.

3.3 Relationship Redesign

3.3.1 Reservation & Service

Business Rules:

- Each reservation must refer to one service.
- Each service can be associated with multiple reservations.

Assumptions:

- A reservation cannot exist without a service.

3.3.2 Service & Package Service

Business Rules:

- A service can be included in multiple package services.
- Each package service refers to one service.

Assumptions:

- Package Service acts as a bridge entity.

3.3.3 Tour Package & Package Service

Business Rules:

- A tour package can contain multiple services.
- Each package service belongs to one tour package.

Assumptions:

- Services can be reused across multiple packages.

3.3.4 Supplier & Service

Business Rules:

- Each supplier provides one or more services.
- Each service must be provided by exactly one supplier.

Assumptions:

- Supplier specialization determines the type of service provided.

3.3.5 Flight Reservation & Ticket

Business Rules:

- A ticket is generated for a confirmed flight reservation.
- Each ticket must be associated with exactly one flight reservation.
- A flight reservation can generate one or more tickets depending on the number of passengers.

Assumptions:

- Ticket issuance occurs only after successful booking confirmation and payment completion.
- Only flight reservations generate tickets; hotel and transport reservations do not require tickets in this system.
- Each ticket contains specific details such as seat number and ticket status.

4.0 Data Dictionary

4.1 Customer

Attribute	Data Type	Size	Constraint	Description
customer_ID	CHAR	7	PRIMARY KEY	Customer ID
first_name	VARCHAR2	50	NOT NULL	Customer's first name
last_name	VARCHAR2	50	NOT NULL	Customer's last name
email	VARCHAR2	100	UNIQUE KEY	Customer email address
address	VARCHAR2	225		Customer address
phone_NO	VARCHAR2	15	NOT NULL	Contact number
passport_NO	VARCHAR2	20	UNIQUE KEY, NOT NULL	Passport number

4.2 Tour_Package

Attribute	Data Type	Size	Constraint	Description
package_ID	CHAR	6	PRIMARY KEY	Package ID
package_type	VARCHAR2	50	NOT NULL	Type of package
package_description	VARCHAR2	200		Description
duration	VARCHAR2	50	NOT NULL	Duration of tour
price	NUMBER	8, 2	NOT NULL	Package price

4.3 Booking

Attribute	Data Type	Size	Constraint	Description
booking_ID	CHAR	6	PRIMARY KEY	Booking ID
customer_ID	CHAR	7	FOREIGN KEY, NOT NULL	Customer reference
package_ID	CHAR	6	FOREIGN KEY, NOT NULL	Package booked
booking_date	DATE		NOT NULL	Booking date
booking_status	VARCHAR2	20	CHECK CONDITION, NOT NULL	Status
total_price	NUMBER	8, 2	NOT NULL	Total cost

4.4 Itinerary

Attribute	Data Type	Size	Constraint	Description
itinerary_ID	CHAR	6	PRIMARY KEY	Itinerary ID
package_ID	CHAR	6	FOREIGN KEY, NOT NULL	The associated tour package
day_NO	NUMBER	2	CHECK CONDITION, NOT NULL	Number of days
activity	VARCHAR	200		Activity planned
location	VARCHAR2	200		Location
time	VARCHAR2	8		Time

4.5 Tour Guide

Attribute	Data Type	Size	Constraint	Description
guide_ID	CHAR	6	PRIMARY KEY	Tour guide ID
staff_first_name	VARCHAR2	100	NOT NULL	first name of tour guide
staff_last_name	VARCHAR2	100	NOT NULL	last name of tour guide
staff_contact_NO	VARCHAR2	15	NOT NULL	Contact number of guides
language	VARCHAR2	100		Languages spoken

4.6 Package Guide

Attribute	Data Type	Size	Constraint	Description
package_ID	CHAR	6	PRIMARY KEY, FOREIGN KEY	Tour package ID
guide_ID	CHAR	6	PRIMARY KEY, FOREIGN KEY	Assigned tour guide ID
assignment_role	VARCHAR2	50	NOT NULL	Role of the guide
start_date	DATE		CHECK CONDITION, NOT NULL	Assignment start date
end_date	DATE		CHECK CONDITION, NOT NULL	Assignment end date

4.7 Ticket

Attribute	Data Type	Size	Constraint	Description
ticket_ID	CHAR	6	PRIMARY KEY	Ticket ID
reservation_ID	CHAR	6	FOREIGN KEY, NOT NULL	Associated reservation ID
ticket_NO	CHAR	11	UNIQUE KEY, NOT NULL	Ticket number
seat_NO	VARCHAR2	5	NOT NULL	Seat number
issue_date	DATE			The date of issue
ticket_status	VARCHAR2	20	CHECK CONDITION	Status of ticket

4.8 Payment

Attribute	Data Type	Size	Constraint	Description
payment_ID	CHAR	6	PRIMARY KEY	Payment ID
booking_ID	CHAR	6	FOREIGN KEY, NOT NULL	Related booking
payment_amount	NUMBER	8, 2	CHECK CONDITION, NOT NULL	Amount paid
payment_method	VARCHAR2	30	CHECK CONDITION, NOT NULL	Payment method
payment_datetime	TIMESTAMP			Date & time
payment_status	VARCHAR2	20	CHECK CONDITION, NOT NULL	Payment status

4.9 Supplier

Attribute	Data Type	Size	Constraint	Description
supplier_ID	CHAR	6	PRIMARY KEY	Supplier ID
supplier_name	VARCHAR2	100	UNIQUE KEY, NOT NULL	Supplier name
contact_number_NO	VARCHAR2	15	NOT NULL	Contact Number of Supplier
email	VARCHAR2	50	UNIQUE KEY	Email of each supplier

4.10 Airline

Attribute	Data Type	Size	Constraint	Description
supplier_ID	CHAR	6	PRIMARY KEY, FOREIGN KEY	Associated Supplier ID
airline_name	VARCHAR2	100	NOT NULL	Airline Name
iata_code	CHAR	2	UNIQUE KEY, NOT NULL	International Air Transport Association (IATA) code

4.11 Hotel_Provider

Attribute	Data Type	Size	Constraint	Description
supplier_ID	CHAR	6	PRIMARY KEY, FOREIGN KEY	Associated Supplier ID
hotel_name	VARCHAR2	100	NOT NULL	Hotel name
star_rating	NUMBER	2,1		Hotel rating

4.12 Transport_Provider

Attribute	Data Type	Size	Constraint	Description
supplier_ID	CHAR	6	PRIMARY KEY, FOREIGN KEY	Associated Supplier ID
company_name	VARCHAR2	100	NOT NULL	Transport company name

4.13 Service

Attribute	Data Type	Size	Constraint	Description
service_ID	CHAR	6	PRIMARY KEY	Service ID
supplier_ID	CHAR	6	FOREIGN KEY, NOT NULL	Associated Supplier ID
service_name	VARCHAR2	100	NOT NULL	Name of the service
service_price	NUMBER	8, 2	NOT NULL	Price of the service

4.14 Flight

Attribute	Data Type	Size	Constraint	Description
service_ID	CHAR	6	PRIMARY KEY, FOREIGN KEY	Associated flight service ID
flight_number	VARCHAR2	10	UNIQUE KEY, NOT NULL	Unique flight number
departure_airport	VARCHAR2	10	NOT NULL	Departure airport code
arrival_airport	VARCHAR2	10	NOT NULL	Arrival airport code
departure_time	TIMESTAMP		NOT NULL	Flight departure date and time
arrival_time	TIMESTAMP		NOT NULL	Flight arrival date and time
duration	VARCHAR2	20	NOT NULL	Duration of flight
boarding_gate	VARCHAR2	3	NOT NULL	Boarding gate number

4.15 Hotel

Attribute	Data Type	Size	Constraint	Description
service_ID	CHAR	6	PRIMARY KEY, FOREIGN KEY	Associated hotel service ID
hotel_name	VARCHAR2	100	NOT NULL	Name of the hotel
location	VARCHAR2	100	NOT NULL	Location of the hotel
room_type	VARCHAR2	50	NOT NULL	Type of room
price_per_night	NUMBER	8, 2	NOT NULL	Cost per night

4.16 Transport

Attribute	Data Type	Size	Constraint	Description
service_ID	CHAR	6	PRIMARY KEY, FOREIGN KEY	Associated transport service ID
vehicle_type	VARCHAR2	50	NOT NULL	Type of transport
capacity	NUMBER	4	NOT NULL	Maximum number of passengers
base_location	VARCHAR2	100	NOT NULL	Starting/base location of transport

4.17 Reservation

Attribute	Data Type	Size	Constraint	Description
reservation_ID	CHAR	6	PRIMARY KEY	Reservation ID
booking_ID	CHAR	6	FOREIGN KEY, NOT NULL	Related booking ID
reservation_date	DATE			Date when the reservation was made
reservation_status	VARCHAR2	20	CHECK CONDITION, NOT NULL	Status of reservation

4.18 Flight Reservation

Attribute	Data Type	Size	Constraint	Description
reservation_ID	CHAR	6	PRIMARY KEY, FOREIGN KEY	Associated reservation ID
service_ID	CHAR	6	FOREIGN KEY, NOT NULL	Associated service ID
seat_class	VARCHAR2	20	CHECK CONDITION, NOT NULL	Class of seat
seat_NO	VARCHAR2	3	NOT NULL	Seat number assigned

4.19 Hotel Reservation

Attribute	Data Type	Size	Constraint	Description
reservation_ID	CHAR	6	PRIMARY KEY, FOREIGN KEY	Associated reservation ID
service_ID	CHAR	6	FOREIGN KEY	Associated service ID
check_in_date	DATE		CHECK CONDITION, NOT NULL	Date of check-in
check_out_date	DATE		CHECK CONDITION, NOT NULL	Date of check-out
num_of_rooms	NUMBER	2	NOT NULL	Number of rooms booked

4.20 Transport Reservation

Attribute	Data Type	Size	Constraint	Description
reservation_ID	CHAR	6	PRIMARY KEY, FOREIGN KEY	Associated reservation ID
service_ID	CHAR	6	FOREIGN KEY, NOT NULL	Associated service ID
pick_up_location	VARCHAR2	100	NOT NULL	Pickup location
drop_off_location	VARCHAR2	100	NOT NULL	Drop-off location
pick_up_datetime	TIMESTAMP		NOT NULL	Pickup date and time
drop_off_datetime	TIMESTAMP		NOT NULL	Drop-off date and time

4.21 Package Service

Attribute	Data Type	Size	Constraint	Description
package_ID	CHAR	6	PRIMARY KEY, FOREIGN KEY	Associated Package ID
service_ID	CHAR	6	PRIMARY KEY, FOREIGN KEY	Associated Service ID
custom_price	NUMBER	8, 2	NOT NULL	Custom Price
sequence_NO	NUMBER	3	NOT NULL	Number of the sequences

5.0 Tables and Records

5.1 Table Customer

customer_ID	first_name	last_name	email	address	phone_NO	passport_NO
CUST001	Alice	Johnson	alice.j@email.com	123 Jalan Ampang	012-3456789	A12345678
CUST002	Bernard	Lee	bernard.l@email.com	45 Jalan Penang	013-4567890	B23456789
CUST003	Chloe	Tan	chloe.t@email.com	88 Jalan Gaya	014-5678901	C34567890
CUST004	David	Raju	david.r@email.com	12 Jalan JB	015-6789012	D45678901
CUST005	Eva	Wong	eva.w@email.com	99 Jalan Kuching	016-7890123	E56789012

5.2 Table Tour Package

package_ID	package_type	package_description	duration	package_price
PKG001	Adventure	Mount Kinabalu Hiking Expedition	4 Days 3 Nights	1500.00
PKG002	Relaxation	Langkawi Beach Retreat	3 Days 2 Nights	850.00
PKG003	Cultural	Penang Heritage Walk & Food Tour	2 Days 1 Night	450.00
PKG004	Nature	Taman Negara Rainforest Exploration	5 Days 4 Nights	1200.00
PKG005	City	Kuala Lumpur Metropolis Tour	3 Days 2 Nights	600.00

5.3 Table Booking

booking_ID	customer_ID	package_ID	booking_date	booking_status	total_price
BKG001	CUST001	PKG001	10-MAY-2025	Confirmed	1,500.00
BKG002	CUST002	PKG002	12-MAY-2025	Confirmed	850.00
BKG003	CUST003	PKG003	15-MAY-2025	Pending	450.00
BKG004	CUST004	PKG004	18-MAY-2025	Confirmed	1,200.00
BKG005	CUST005	PKG005	20-MAY-2025	Cancelled	600.00

5.4 Table Itinerary

Itinerary_ID	Package_ID	day_NO	activity	location	time
ITN001	PKG001	1	Arrival and Base Camp Briefing	Kinabalu Park HQ	14:00
ITN002	PKG001	2	Ascent to Panalaban	Mount Kinabalu Trail	8:00
ITN003	PKG002	1	Island Hopping	Langkawi Archipelago	09:00
ITN004	PKG003	1	Street Art Walk	Georgetown, Penang	10:30
ITN005	PKG005	2	Petronas Twin Towers Visit	KLCC	11:00

5.5 Table Tour_Guide

guide_ID	staff_first_name	staff_last_name	staff_contact_NO	language
GDE001	Ahmad	Faizal	019-1112222	English, Malay
GDE002	Sarah	Lim	012-3334444	English, Mandarin
GDE003	Muthu	Kumar	017-5556666	English, Tamil
GDE004	Diana	Rose	016-7778888	English, French
GDE005	Kenji	Sato	014-9990000	English, Japanese

5.6 Table Package_Guide

package_ID	guide_ID	assignment_role	start_date	end_date
PKG001	GDE001	Lead Guide	01-JUN-2025	04-JUN-2025
PKG002	GDE002	Local Escort	02-JUN-2025	04-JUN-2025
PKG003	GDE003	Heritage Specialist	03-JUN-2025	04-JUN-2025
PKG004	GDE004	Nature Guide	04-JUN-2025	08-JUN-2025
PKG005	GDE005	City Translator	05-JUN-2025	07-JUN-2025

5.7 Table Ticket

ticket_id	reservation_id	ticket_no	seat_NO	issue_date	ticket_status
TCK001	RES001	ETKT-990122	12A	11-MAY-2025	Issued
TCK002	RES002	ETKT-990123	14C	13-MAY-2025	Issued
TCK003	RES003	ETKT-990124	2B	16-MAY-2025	Issued
TCK004	RES004	ETKT-990125	8F	19-MAY-2025	Issued
TCK005	RES005	ETKT-990126	1A	21-MAY-2025	Cancelled

5.8 Table Payment

payment_ID	booking_ID	payment_amount	payment_method	payment_datetime	payment_status
PAY001	BKG001	1,500.00	Credit Card	10-MAY-25 10:30:00	Success
PAY002	BKG002	850.00	Online Banking	12-MAY-25 14:15:00	Success
PAY003	BKG003	450.00	E-Wallet	15-MAY-25 09:00:00	Pending
PAY004	BKG004	1,200.00	Credit Card	18-MAY-25 16:45:00	Success
PAY005	BKG005	600.00	Online Banking	20-MAY-25 11:20:00	Refunded

5.9 Table Supplier**i) Airline Suppliers**

supplier_ID	supplier_name	supplier_contact_NO	email
SUP001	Malaysia Airlines	1-300-88-3000	mh@airline.com
SUP002	AirAsia	1-300-88-9933	support@airasia.com
SUP003	Batik Air	03-7841-5388	info@batikair.com
SUP004	Firefly	03-7845-4543	customer@fireflyz.com
SUP005	Singapore Airlines	1-800-80-3333	contact@singaporeair.com

ii) Hotel Suppliers

supplier_ID	supplier_name	supplier_contact_NO	email
SUP006	Shangri-La Hotels	03-2032-2388	reservations@shangri-la.com
SUP007	Hilton Worldwide	03-2264-2264	info@hilton.com
SUP008	Marriott International	03-2715-9000	booking@marriott.com
SUP009	Tune Hotels	03-2082-5600	hello@tunehotels.com
SUP010	Banyan Tree	103-2113-1888	kl@banyantree.com

iii) Transport Suppliers

supplier_ID	supplier_name	supplier_contact_NO	email
SUP011	Aeroline	03-6258-8800	enquiry@aeroline.com.my
SUP012	Plusliner	03-2072-8888	customercare@plusliner.com
SUP013	Transnasional	1-300-88-8582	info@transnasional.com
SUP014	KLIA Ekspres	03-2267-8000	customerenquiry@kliaekspres.com
SUP015	RapidKL Bus Services	03-7885-2585	suggest@rapidkl.com.my

5.10 Table Airline

supplier_ID	airline_name	iata_code
SUP001	Malaysia Airlines	MH
SUP002	AirAsia	AK
SUP003	Batik Air	OD
SUP004	Firefly	FY
SUP005	Singapore Airlines	SQ

5.11 Table Hotel_Provider

supplier_ID	hotel_name	star_rating
SUP006	Shangri-La Kuala Lumpur	5.0
SUP007	Hilton Petaling Jaya	4.5
SUP008	JW Marriott KL	5.0
SUP009	Tune Hotel KLIA2	3.0
SUP010	Banyan Tree KL	5.0

5.12 Table Transport_Provider

supplier_ID	company_name
SUP011	Aeroline
SUP012	Plusliner
SUP013	Transnasional
SUP014	KLIA Ekspres
SUP015	RapidKL

5.13 Table Service**i)Flight Services**

service_ID	supplier_ID	service_name	service_price
SRV001	SUP001	Flight KUL-BKI	400.00
SRV002	SUP002	Flight KUL-LGK	150.00
SRV003	SUP003	Flight KUL-PEN	120.00
SRV004	SUP004	Flight SZB-KBR	180.00
SRV005	SUP005	Flight KUL-SIN	500.00

ii) Hotel Services

service_ID	supplier_ID	service_name	service_price
SRV006	SUP006	Deluxe Stay	550.00
SRV007	SUP007	Exec Stay	650.00
SRV008	SUP008	Standard Stay	480.00
SRV009	SUP009	Budget Stay	150.00
SRV010	SUP010	Suite Stay	1200.00

iii) Transport Services

service_ID	supplier_ID	service_name	service_price
SRV011	SUP011	Coach Transfer KL	80.00
SRV012	SUP012	Bus Transfer Penang	40.00
SRV013	SUP013	Express Bus JB	45.00
SRV014	SUP014	Airport Train	55.00
SRV015	SUP015	City Bus Pass	10.00

5.14 Table Flight

service_ID	Flight_number	departur e_airport	arrival_ airport	departur e_time	arrival_ time	durat ion	boardin g_gate
SRV001	MH2604	KUL	BKI	01-JUN- 25 08:00:00	01-JUN- 25 10:30:00	2h 30m	G1
SRV002	AK6112	KUL	LGK	02-JUN- 25 09:15:00	02-JUN- 25 10:15:00	1h 00m	H4
SRV003	OD2103	KUL	PEN	03-JUN- 25 14:00:00	03-JUN- 25 15:00:00	1h 00m	K2
SRV004	FY1002	SZB	KBR	04-JUN- 25 11:30:00	04-JUN- 25 12:45:00	1h 15m	A1
SRV005	SQ107	KUL	SIN	05-JUN- 25 18:45:00	05-JUN- 25 19:45:00	1h 00m	C3

5.15 Table Hotel

service_ID	hotel_name	location	room_type	price_per_night
SRV006	Shangri-La KL	Kuala Lumpur	Deluxe Room	550.00
SRV007	Hilton PJ	Petaling Jaya	Executive Suite	650.00
SRV008	JW Marriott	Bukit Bintang	Standard Room	480.00
SRV009	Tune Hotel	KLIA2	Twin Room	150.00
SRV010	Banyan Tree	KLCC	Signature Suite	1200.00

5.16 Table Transport

service_ID	vehicle_type	capacity	base_location
SRV011	Luxury Coach	30	Corus Hotel KL
SRV012	Express Bus	40	TBS KL
SRV013	Express Bus	44	Penang Sentral
SRV014	Train	200	KL Sentral
SRV015	City Bus	60	Pasar Seni Bus Hub

5.17 Table Reservation**i) For Flights**

reservation_ID	booking_ID	reservation_date	reservation_status
RES001	BKG001	10-MAY-2025	Confirmed
RES002	BKG002	12-MAY-2025	Confirmed
RES003	BKG003	15-MAY-2025	Pending
RES004	BKG004	18-MAY-2025	Confirmed
RES005	BKG005	20-MAY-2025	Cancelled

ii) For Hotels

reservation_ID	booking_ID	reservation_date	reservation_status
RES006	BKG001	10-MAY-2025	Confirmed
RES007	BKG002	12-MAY-2025	Confirmed
RES008	BKG003	15-MAY-2025	Pending
RES009	BKG004	18-MAY-2025	Confirmed
RES010	BKG005	20-MAY-2025	Cancelled

iii) For Transports

reservation_ID	booking_ID	reservation_date	reservation_status
RES011	BKG001	10-MAY-2025	Confirmed
RES012	BKG002	12-MAY-2025	Confirmed
RES013	BKG003	15-MAY-2025	Pending
RES014	BKG004	18-MAY-2025	Confirmed
RES015	BKG005	20-MAY-2025	Cancelled

5.18 Table Flight Reservation

reservation_ID	service_ID	seat_class
RES001	SRV001	Economy
RES002	SRV002	Economy
RES003	SRV003	Business
RES004	SRV004	Economy
RES005	SRV005	First Class

5.19 Table Hotel Reservation

reservation_ID	service_ID	check_in_date	check_out_date	num_of_rooms
RES006	SRV006	01-JUN-2025	04-JUN-2025	1
RES007	SRV007	02-JUN-2025	04-JUN-2025	2
RES008	SRV008	03-JUN-2025	04-JUN-2025	1
RES009	SRV009	04-JUN-2025	08-JUN-2025	3
RES010	SRV010	05-JUN-2025	07-JUN-2025	1

5.20 Table Transport Reservation

reservation_ID	service_ID	pick_up_location	drop_off_location	pick_up_datetime	drop_off_datetime
RES011	SRV011	KL Sentral	Mount Kinabalu Base	01-JUN-25 08:00:00	01-JUN-25 12:00:00
RES012	SRV012	Penang Airport	Langkawi Jetty	02-JUN-25 09:00:00	02-JUN-25 11:00:00
RES013	SRV013	Hotel Lobby	Heritage Zone	03-JUN-25 10:00:00	03-JUN-25 18:00:00
RES014	SRV014	KLIA	Taman Negara	04-JUN-25 14:00:00	04-JUN-25 17:00:00
RES015	SRV015	Bukit Bintang	Batu Caves	05-JUN-25 08:30:00	05-JUN-25 13:00:00

5.21 Table Package_Service

package_ID	service_ID	custom_price	sequence_NO
PKG001	SRV001	350.00	1
PKG001	SRV006	500.00	2
PKG002	SRV002	120.00	1
PKG002	SRV009	130.00	2
PKG003	SRV012	35.00	1

6.0 Script

```

Rem
Rem  NAME
Rem    diy_holiday.sql - Create data objects for DIY Holiday schema
Rem
Rem  DESCRIPTION
Rem    This script creates 21 tables, associated constraints,
Rem    inserts sample data and applies terminal display formatting.
Rem

```

```

REM Before creating the tables
REM drop all tables first

```

```

-- Children first, Parents last
DROP TABLE ticket CASCADE CONSTRAINTS;
DROP TABLE flight_reservation CASCADE CONSTRAINTS;
DROP TABLE hotel_reservation CASCADE CONSTRAINTS;
DROP TABLE transport_reservation CASCADE CONSTRAINTS;
DROP TABLE transport CASCADE CONSTRAINTS;
DROP TABLE hotel CASCADE CONSTRAINTS;
DROP TABLE flight CASCADE CONSTRAINTS;
DROP TABLE package_service CASCADE CONSTRAINTS;
DROP TABLE package_guide CASCADE CONSTRAINTS;
DROP TABLE payment CASCADE CONSTRAINTS;
DROP TABLE reservation CASCADE CONSTRAINTS;
DROP TABLE booking CASCADE CONSTRAINTS;
DROP TABLE service CASCADE CONSTRAINTS;
DROP TABLE itinerary CASCADE CONSTRAINTS;
DROP TABLE airline CASCADE CONSTRAINTS;
DROP TABLE hotel_provider CASCADE CONSTRAINTS;
DROP TABLE transport_provider CASCADE CONSTRAINTS;
DROP TABLE tour_guide CASCADE CONSTRAINTS;
DROP TABLE tour_package CASCADE CONSTRAINTS;
DROP TABLE customer CASCADE CONSTRAINTS;
DROP TABLE supplier CASCADE CONSTRAINTS;

```

```

SET DEFINE OFF;

```

```

REM *****
REM Apply Terminal Display Formatting (For neat SELECT/DESCRIBE outputs)
REM *****
SET LINESIZE 138;
SET PAGESIZE 100;
SET WRAP ON;

```

```

ALTER SESSION SET NLS_TIMESTAMP_FORMAT = 'DD-MON-RR HH24:MI:SS';
ALTER SESSION SET NLS_DATE_FORMAT = 'DD-MON-YYYY';

```

```

-- COMMON IDS
COLUMN customer_id FORMAT A12;

```

```
COLUMN package_id FORMAT A10;
COLUMN guide_id FORMAT A10;
COLUMN service_id FORMAT A10;
COLUMN supplier_id FORMAT A12;
COLUMN booking_id FORMAT A10;
COLUMN reservation_id FORMAT A14;
COLUMN payment_id FORMAT A10;
COLUMN ticket_id FORMAT A10;

-- CUSTOMER
COLUMN first_name FORMAT A15;
COLUMN last_name FORMAT A15;
COLUMN email FORMAT A30;
COLUMN address FORMAT A30;
COLUMN phone_no FORMAT A15;
COLUMN passport_no FORMAT A15;

-- TOUR GUIDE
COLUMN staff_first_name FORMAT A15;
COLUMN staff_last_name FORMAT A15;
COLUMN staff_contact_no FORMAT A15;
COLUMN language FORMAT A25;

-- TOUR PACKAGE
COLUMN package_type FORMAT A15;
COLUMN package_price FORMAT 999,999.00;
COLUMN package_description FORMAT A40;
COLUMN duration FORMAT A20;

-- PACKAGE GUIDE
COLUMN assignment_role FORMAT A20;
COLUMN start_date FORMAT A12;
COLUMN end_date FORMAT A12;

-- PAYMENT
COLUMN payment_amount FORMAT 999,999.00;
COLUMN payment_method FORMAT A20;
COLUMN payment_datetime FORMAT A20;
COLUMN payment_status FORMAT A12;

-- TICKET
COLUMN ticket_no FORMAT A15;
COLUMN seat_no FORMAT A8;
COLUMN issue_date FORMAT A12;
COLUMN ticket_status FORMAT A12;

-- SUPPLIER
COLUMN supplier_name FORMAT A25;
COLUMN company_name FORMAT A25;
COLUMN supplier_contact_no FORMAT A20;
```

```

COLUMN email FORMAT A35;

-- SERVICE
COLUMN service_name FORMAT A25;
COLUMN service_price FORMAT 999,999.00;
COLUMN custom_price FORMAT 999,999.00;

-- AIRLINE
COLUMN airline_name FORMAT A25;
COLUMN iata_code FORMAT A8;

-- FLIGHT
COLUMN flight_number FORMAT A10;
COLUMN departure_airport FORMAT A10;
COLUMN arrival_airport FORMAT A10;
COLUMN departure_time FORMAT A20;
COLUMN arrival_time FORMAT A20;
COLUMN duration FORMAT A10;
COLUMN boarding_gate FORMAT A8;

-- HOTEL
COLUMN hotel_name FORMAT A25;
COLUMN location FORMAT A25;
COLUMN room_type FORMAT A20;
COLUMN price_per_night FORMAT 999,999.00;

-- TRANSPORT
COLUMN vehicle_type FORMAT A20;
COLUMN capacity FORMAT A25;
COLUMN base_location FORMAT A25;

-- TRANSPORT RESERVATION
COLUMN pick_up_location FORMAT A20;
COLUMN drop_off_location FORMAT A25;
COLUMN pick_up_datetime FORMAT A20;
COLUMN drop_off_datetime FORMAT A20;

-- BOOKING
COLUMN booking_date FORMAT A20;
COLUMN total_price FORMAT 999,999.00;
COLUMN booking_status FORMAT A15;

-- RESERVATION
COLUMN reservation_date FORMAT A15;
COLUMN reservation_status FORMAT A20;

-- ITINERARY
COLUMN activity FORMAT A35;
COLUMN time FORMAT A10;
COLUMN location FORMAT A30;

```

-- STAR RATING

COLUMN star_rating FORMAT 9.9;

REM *****

REM Create the CUSTOMER table

Prompt ***** Creating CUSTOMER table

```
CREATE TABLE customer
( customer_id  CHAR(7)
, first_name   VARCHAR2(50)
  CONSTRAINT cust_fname_nn NOT NULL
, last_name    VARCHAR2(50)
  CONSTRAINT cust_lname_nn NOT NULL
, email        VARCHAR2(100)
, address      VARCHAR2(225)
, phone_no     VARCHAR2(15)
  CONSTRAINT cust_phone_no_nn NOT NULL
, passport_no  VARCHAR2(20)
  CONSTRAINT cust_passport_no_nn NOT NULL
);
```

```
ALTER TABLE customer
ADD ( CONSTRAINT cust_id_pk
      PRIMARY KEY (customer_id)
, CONSTRAINT cust_email_uk
  UNIQUE (email)
, CONSTRAINT cust_passport_no_uk
  UNIQUE (passport_no)
);
```

REM *****

REM Create the TOUR_PACKAGE table

Prompt ***** Creating TOUR_PACKAGE table

```
CREATE TABLE tour_package
( package_id    CHAR(6)
, package_type  VARCHAR2(50)
  CONSTRAINT pkg_type_nn NOT NULL
, package_description VARCHAR2(200)
, duration      VARCHAR2(50)
  CONSTRAINT pkg_duration_nn NOT NULL
, package_price NUMBER(8, 2)
  CONSTRAINT pkg_price_nn NOT NULL
);
```

```
ALTER TABLE tour_package
ADD ( CONSTRAINT pkg_id_pk
```

```

        PRIMARY KEY (package_id)
    );

```

```

REM *****
REM Create the SUPPLIER table (Supertype)

```

Prompt ***** Creating SUPPLIER table

```

CREATE TABLE supplier
( supplier_id      CHAR(6)
, supplier_name    VARCHAR2(100)
  CONSTRAINT supp_name_nn NOT NULL
, supplier_contact_no VARCHAR2(15)
, email            VARCHAR2(50)
);

```

```

ALTER TABLE supplier
ADD ( CONSTRAINT supp_id_pk
      PRIMARY KEY (supplier_id)
, CONSTRAINT supp_name_uk
  UNIQUE (supplier_name)
, CONSTRAINT supp_email_uk
  UNIQUE (email)
);

```

```

REM *****
REM Create the TOUR_GUIDE table

```

Prompt ***** Creating TOUR_GUIDE table

```

CREATE TABLE tour_guide
( guide_id      CHAR(6)
, staff_first_name VARCHAR2(100)
  CONSTRAINT staff_fname_nn NOT NULL
, staff_last_name  VARCHAR2(100)
  CONSTRAINT staff_lname_nn NOT NULL
, staff_contact_no VARCHAR2(15)
  CONSTRAINT staff_contact_nn NOT NULL
, language         VARCHAR2(100)
);

```

```

ALTER TABLE tour_guide
ADD ( CONSTRAINT tguide_id_pk
      PRIMARY KEY (guide_id)
);

```

```

REM *****
REM Create the AIRLINE table (Subtype of Supplier)

```

Prompt ***** Creating AIRLINE table


```
CREATE TABLE airline
( supplier_id CHAR(6)
, airline_name VARCHAR2(100)
  CONSTRAINT air_name_nn NOT NULL
, iata_code CHAR(2)
  CONSTRAINT iata_code_nn NOT NULL
);
```

```
ALTER TABLE airline
ADD ( CONSTRAINT airline_id_pk
      PRIMARY KEY (supplier_id)
, CONSTRAINT airline_supp_fk
      FOREIGN KEY (supplier_id)
      REFERENCES supplier (supplier_id)
, CONSTRAINT air_iata_uk
      UNIQUE (iata_code)
);
```

```
REM *****
REM Create the HOTEL_PROVIDER table (Subtype of Supplier)
```

Prompt ***** Creating HOTEL_PROVIDER table

```
CREATE TABLE hotel_provider
( supplier_id CHAR(6)
, hotel_name VARCHAR2(100)
  CONSTRAINT hprov_name_nn NOT NULL
, star_rating NUMBER(2, 1)
);
```

```
ALTER TABLE hotel_provider
ADD ( CONSTRAINT hprov_id_pk
      PRIMARY KEY (supplier_id)
, CONSTRAINT hprov_supp_fk
      FOREIGN KEY (supplier_id)
      REFERENCES supplier (supplier_id)
);
```

```
REM *****
REM Create the TRANSPORT_PROVIDER table (Subtype of Supplier)
```

Prompt ***** Creating TRANSPORT_PROVIDER table

```
CREATE TABLE transport_provider
( supplier_id CHAR(6)
, company_name VARCHAR2(100)
  CONSTRAINT tprov_name_nn NOT NULL
);
```

```
ALTER TABLE transport_provider
ADD ( CONSTRAINT tprov_id_pk
      PRIMARY KEY (supplier_id)
    , CONSTRAINT tprov_supp_fk
      FOREIGN KEY (supplier_id)
      REFERENCES supplier (supplier_id)
    );
```

REM *****
 REM Create the SERVICE table (Supertype)

Prompt ***** Creating SERVICE table

```
CREATE TABLE service
( service_id CHAR(6)
, supplier_id CHAR(6)
  CONSTRAINT srv_supp_nn NOT NULL
, service_name VARCHAR2(100)
  CONSTRAINT srv_name_nn NOT NULL
, service_price NUMBER(8, 2)
  CONSTRAINT srv_price_nn NOT NULL
);
```

```
ALTER TABLE service
ADD ( CONSTRAINT srv_id_pk
      PRIMARY KEY (service_id)
    , CONSTRAINT srv_supp_fk
      FOREIGN KEY (supplier_id)
      REFERENCES supplier (supplier_id)
    );
```

REM *****
 REM Create the BOOKING table

Prompt ***** Creating BOOKING table

```
CREATE TABLE booking
( booking_id CHAR(6)
, customer_id CHAR(7)
  CONSTRAINT bkg_cust_nn NOT NULL
, package_id CHAR(6)
  CONSTRAINT pkg_pac_nn NOT NULL
, booking_date DATE DEFAULT SYSDATE
  CONSTRAINT bk_date_nn NOT NULL
, booking_status VARCHAR2(20)
  CONSTRAINT bkg_status_nn NOT NULL
, total_price NUMBER(8, 2)
  CONSTRAINT bkg_price_nn NOT NULL
);
```

```
ALTER TABLE booking
ADD ( CONSTRAINT bkg_id_pk
      PRIMARY KEY (booking_id)
    , CONSTRAINT bkg_cust_fk
      FOREIGN KEY (customer_id)
      REFERENCES customer (customer_id)
    , CONSTRAINT bkg_pkg_fk
      FOREIGN KEY (package_id)
      REFERENCES tour_package (package_id)
    , CONSTRAINT bkg_status_ck
      CHECK (booking_status IN ('Confirmed','Pending','Cancelled'))
    );
```

REM *****

REM Create the ITINERARY table

Prompt ***** Creating ITINERARY table

```
CREATE TABLE itinerary
( itinerary_id CHAR(6)
, package_id CHAR(6)
  CONSTRAINT itin_pkg_nn NOT NULL
, day_no NUMBER(2)
, activity VARCHAR2(200)
, location VARCHAR2(200)
, time VARCHAR2(8)
);
```

```
ALTER TABLE itinerary
ADD ( CONSTRAINT itin_id_pk
      PRIMARY KEY (itinerary_id)
    , CONSTRAINT itin_pkg_fk
      FOREIGN KEY (package_id)
      REFERENCES tour_package (package_id)
    , CONSTRAINT day_min
      CHECK (day_no > 0)
    );
```

REM *****

REM Create the FLIGHT table (Subtype of Service)

Prompt ***** Creating FLIGHT table

```
CREATE TABLE flight
( service_id CHAR(6)
, flight_number VARCHAR2(10)
  CONSTRAINT flight_num_nn NOT NULL
, departure_airport VARCHAR2(10)
  CONSTRAINT dept_airport_nn NOT NULL
, arrival_airport VARCHAR2(10)
```

```

CONSTRAINT arr_airport_nn NOT NULL
, departure_time  TIMESTAMP
CONSTRAINT dept_time_nn NOT NULL
, arrival_time    TIMESTAMP
CONSTRAINT arr_time_nn NOT NULL
, duration        VARCHAR2(20)
CONSTRAINT duration_nn NOT NULL
, boarding_gate   VARCHAR2(3)
CONSTRAINT board_gate_nn NOT NULL
);

```

```

ALTER TABLE flight
ADD ( CONSTRAINT flight_id_pk
      PRIMARY KEY (service_id)
    , CONSTRAINT flight_srv_fk
      FOREIGN KEY (service_id)
        REFERENCES service (service_id)
    , CONSTRAINT flight_no_uk
      UNIQUE (flight_number)
    );

```

```

REM *****
REM Create the HOTEL table (Subtype of Service)

```

Prompt ***** Creating HOTEL table

```

CREATE TABLE hotel
( service_id  CHAR(6)
, hotel_name  VARCHAR2(100)
CONSTRAINT htl_name_nn NOT NULL
, location    VARCHAR2(100)
CONSTRAINT location_nn NOT NULL
, room_type   VARCHAR2(50)
CONSTRAINT rm_type_nn NOT NULL
, price_per_night NUMBER(8, 2)
CONSTRAINT price_per_night_nn NOT NULL
);

```

```

ALTER TABLE hotel
ADD ( CONSTRAINT hotel_id_pk
      PRIMARY KEY (service_id)
    , CONSTRAINT hotel_srv_fk
      FOREIGN KEY (service_id)
        REFERENCES service (service_id)
    );

```

```

REM *****
REM Create the TRANSPORT table (Subtype of Service)

```

Prompt ***** Creating TRANSPORT table

```
CREATE TABLE transport
( service_id CHAR(6)
, vehicle_type VARCHAR2(50)
CONSTRAINT vhcle_type_nn NOT NULL
, capacity NUMBER(4)
CONSTRAINT capacity_nn NOT NULL
, base_location VARCHAR2(100)
CONSTRAINT base_loc_nn NOT NULL
);
```

```
ALTER TABLE transport
ADD ( CONSTRAINT transp_id_pk
      PRIMARY KEY (service_id)
, CONSTRAINT transp_srv_fk
      FOREIGN KEY (service_id)
      REFERENCES service (service_id)
);
```

REM *****
 REM Create the PACKAGE_SERVICE table (Associative Entity)

Prompt ***** Creating PACKAGE_SERVICE table

```
CREATE TABLE package_service
( package_id CHAR(6)
, service_id CHAR(6)
, custom_price NUMBER(8, 2)
CONSTRAINT custom_price_nn NOT NULL
, sequence_no NUMBER(3)
CONSTRAINT seq_no_nn NOT NULL
);
```

```
ALTER TABLE package_service
ADD ( CONSTRAINT pkg_srv_pk
      PRIMARY KEY ( package_id, service_id)
, CONSTRAINT pkg_srv_pkg_fk
      FOREIGN KEY (package_id)
      REFERENCES tour_package (package_id)
, CONSTRAINT pkg_srv_srv_fk
      FOREIGN KEY (service_id)
      REFERENCES service (service_id)
);
```

REM *****
 REM Create the PACKAGE_GUIDE table

Prompt ***** Creating PACKAGE_GUIDE table

```
CREATE TABLE package_guide
```

```
( package_id    CHAR(6)
, guide_id     CHAR(6)
, assignment_role VARCHAR2(50)
CONSTRAINT asgm_role_nn NOT NULL
, start_date   DATE
CONSTRAINT start_dt_nn NOT NULL
, end_date     DATE
CONSTRAINT end_dt_nn NOT NULL
);
```

```
ALTER TABLE package_guide
ADD ( CONSTRAINT pkgguide_pk
      PRIMARY KEY (package_id, guide_id)
, CONSTRAINT pkgguide_pkg_fk
      FOREIGN KEY (package_id)
      REFERENCES tour_package (package_id)
, CONSTRAINT pkgguide_guide_fk
      FOREIGN KEY (guide_id)
      REFERENCES tour_guide (guide_id)
, CONSTRAINT pkgguide_date_ck
      CHECK (end_date >= start_date)
);
```

```
REM *****
REM Create the PAYMENT table
```

Prompt ***** Creating PAYMENT table

```
CREATE TABLE payment
( payment_id    CHAR(6)
, booking_id    CHAR(6)
  CONSTRAINT pay_bkg_nn NOT NULL
, payment_amount NUMBER(8, 2)
  CONSTRAINT pay_amt_nn NOT NULL
, payment_method VARCHAR2(30)
  CONSTRAINT pay_method_nn NOT NULL
, payment_datetime TIMESTAMP DEFAULT SYSDATE
, payment_status VARCHAR2(20)
  CONSTRAINT pay_status_nn NOT NULL
);
```

```
ALTER TABLE payment
ADD ( CONSTRAINT pay_id_pk
      PRIMARY KEY (payment_id)
, CONSTRAINT pay_bkg_fk
      FOREIGN KEY (booking_id)
      REFERENCES booking (booking_id)
, CONSTRAINT pay_method_ck
      CHECK (payment_method IN ('Credit Card','Online Banking','E-Wallet'))
, CONSTRAINT pay_status_ck
```

```

        CHECK (payment_status IN ('Success','Pending','Refunded'))
    , CONSTRAINT pay_amt_ck
        CHECK (payment_amount >= 0)
    );

```

REM *****

REM Create the RESERVATION table (Supertype)

Prompt ***** Creating RESERVATION table

```

CREATE TABLE reservation
( reservation_id  CHAR(6)
, booking_id      CHAR(6)
  CONSTRAINT res_bkg_nn NOT NULL
, reservation_date DATE DEFAULT SYSDATE
, reservation_status VARCHAR2(20)
  CONSTRAINT res_status_nn NOT NULL
);

```

```

ALTER TABLE reservation
ADD ( CONSTRAINT res_id_pk
      PRIMARY KEY (reservation_id)
, CONSTRAINT res_bkg_fk
      FOREIGN KEY (booking_id)
      REFERENCES booking (booking_id)
, CONSTRAINT res_status_ck
      CHECK (reservation_status IN ('Confirmed','Pending','Cancelled'))
);

```

REM *****

REM Create the FLIGHT_RESERVATION table (Subtype of Reservation)

Prompt ***** Creating FLIGHT_RESERVATION table

```

CREATE TABLE flight_reservation
( reservation_id CHAR(6)
, service_id     CHAR(6)
  CONSTRAINT fres_srv_nn NOT NULL
, seat_class    VARCHAR2(20)
  CONSTRAINT seat_cls_nn NOT NULL
);

```

```

ALTER TABLE flight_reservation
ADD ( CONSTRAINT fres_id_pk
      PRIMARY KEY (reservation_id)
, CONSTRAINT fres_res_fk
      FOREIGN KEY (reservation_id)
      REFERENCES reservation (reservation_id)
, CONSTRAINT fres_srv_fk
      FOREIGN KEY (service_id)

```

```

        REFERENCES flight (service_id)
    , CONSTRAINT seat_cls_ck
        CHECK (seat_class IN ('Economy','Business','First Class'))
    );

```

REM *****

REM Create the TICKET table

Prompt ***** Creating TICKET table

```

CREATE TABLE ticket
( ticket_id    CHAR(6)
, reservation_id CHAR(6)
  CONSTRAINT ticket_res_nn NOT NULL
, ticket_no    CHAR(11)
  CONSTRAINT ticket_no_nn NOT NULL
, seat_no     VARCHAR2(5)
  CONSTRAINT seat_no_nn NOT NULL
, issue_date   DATE DEFAULT SYSDATE
, ticket_status VARCHAR2(20)
);

ALTER TABLE ticket
ADD ( CONSTRAINT ticket_id_pk
      PRIMARY KEY (ticket_id)
, CONSTRAINT ticket_res_fk
      FOREIGN KEY (reservation_id)
        REFERENCES flight_reservation (reservation_id)
, CONSTRAINT ticket_no_uk
      UNIQUE (ticket_no)
, CONSTRAINT ticket_status_ck
      CHECK (ticket_status IN ('Issued', 'Used', 'Cancelled'))
);

```

REM *****

REM Create the HOTEL_RESERVATION table (Subtype of Reservation)

Prompt ***** Creating HOTEL_RESERVATION table

```

CREATE TABLE hotel_reservation
( reservation_id CHAR(6)
, service_id    CHAR(6)
, check_in_date  DATE
  CONSTRAINT check_in_nn NOT NULL
, check_out_date DATE
  CONSTRAINT check_out_nn NOT NULL
, num_of_rooms   NUMBER(2)
  CONSTRAINT num_of_rm_nn NOT NULL
);

```



```
ALTER TABLE hotel_reservation
ADD ( CONSTRAINT hres_id_pk
      PRIMARY KEY (reservation_id)
    , CONSTRAINT hres_res_fk
      FOREIGN KEY (reservation_id)
      REFERENCES reservation (reservation_id)
    , CONSTRAINT hres_srv_fk
      FOREIGN KEY (service_id)
      REFERENCES hotel (service_id)
    , CONSTRAINT hres_date_ck
      CHECK (check_out_date >= check_in_date)
    );
```

```
REM *****
REM Create the TRANSPORT_RESERVATION table (Subtype of Reservation)
```

Prompt ***** Creating TRANSPORT_RESERVATION table

```
CREATE TABLE transport_reservation
( reservation_id CHAR(6)
, service_id CHAR(6)
  CONSTRAINT tres_srv_nn NOT NULL
, pick_up_location VARCHAR2(100)
  CONSTRAINT pick_loc_nn NOT NULL
, drop_off_location VARCHAR2(100)
  CONSTRAINT drop_loc_nn NOT NULL
, pick_up_datetime TIMESTAMP
  CONSTRAINT pick_date_nn NOT NULL
, drop_off_datetime TIMESTAMP
  CONSTRAINT drop_date_nn NOT NULL
);
```

```
ALTER TABLE transport_reservation
ADD ( CONSTRAINT tres_id_pk
      PRIMARY KEY (reservation_id)
    , CONSTRAINT tres_res_fk
      FOREIGN KEY (reservation_id)
      REFERENCES reservation (reservation_id)
    , CONSTRAINT tres_srv_fk
      FOREIGN KEY (service_id)
      REFERENCES transport (service_id)
    );
```

```
REM *****insert data into the CUSTOMER table
```

Prompt ***** Populating CUSTOMER table

```
INSERT INTO customer VALUES ( 'CUST001', 'Alice', 'Johnson',
'alice.j@email.com', '123 Jalan Ampang', '012-3456789', 'A12345678');
```

```
INSERT INTO customer VALUES ( 'CUST002', 'Bernard', 'Lee',
'bernard.l@email.com', '45 Jalan Penang', '013-4567890', 'B23456789');
INSERT INTO customer VALUES ( 'CUST003', 'Chloe', 'Tan', 'chloe.t@email.com',
'88 Jalan Gaya', '014-5678901', 'C34567890');
INSERT INTO customer VALUES ( 'CUST004', 'David', 'Raju', 'david.r@email.com',
'12 Jalan JB', '015-6789012', 'D45678901');
INSERT INTO customer VALUES ( 'CUST005', 'Eva', 'Wong', 'eva.w@email.com',
'99 Jalan Kuching', '016-7890123', 'E56789012');
```

REM *****insert data into the TOUR_PACKAGE table

Prompt ***** Populating TOUR_PACKAGE table

```
INSERT INTO tour_package VALUES ( 'PKG001', 'Adventure', 'Mount Kinabalu
Hiking Expedition', '4 Days 3 Nights', 1500.00 );
INSERT INTO tour_package VALUES ( 'PKG002', 'Relaxation', 'Langkawi Beach
Retreat', '3 Days 2 Nights', 850.00 );
INSERT INTO tour_package VALUES ( 'PKG003', 'Cultural', 'Penang Heritage Walk
& Food Tour', '2 Days 1 Nights', 450.00 );
INSERT INTO tour_package VALUES ( 'PKG004', 'Nature', 'Taman Negara
Rainforest Exploration', '5 Days 4 Nights', 1200.00 );
INSERT INTO tour_package VALUES ( 'PKG005', 'City', 'Kuala Lumpur Metropolis
Tour', '3 Days 2 Nights', 600.00 );
```

REM *****insert data into the SUPPLIER table

Prompt ***** Populating SUPPLIER table

```
INSERT INTO supplier VALUES ( 'SUP001', 'Malaysia Airlines', '1-300-88-3000',
'mh@airline.com' );
INSERT INTO supplier VALUES ( 'SUP002', 'AirAsia', '1-300-88-9933',
'support@airasia.com' );
INSERT INTO supplier VALUES ( 'SUP003', 'Batik Air', '03-7841-5388',
'info@batikair.com' );
INSERT INTO supplier VALUES ( 'SUP004', 'Firefly', '03-7845-4543',
'customer@fireflyz.com' );
INSERT INTO supplier VALUES ( 'SUP005', 'Singapore Airlines', '1-800-80-3333',
'contact@singaporeair.com' );
INSERT INTO supplier VALUES ( 'SUP006', 'Shangri-La Hotels', '03-2032-2388',
'reservations@shangri-la.com' );
INSERT INTO supplier VALUES ( 'SUP007', 'Hilton Worldwide', '03-2264-2264',
'info@hilton.com' );
INSERT INTO supplier VALUES ( 'SUP008', 'Marriott International', '03-2715-9000',
'booking@marriott.com' );
INSERT INTO supplier VALUES ( 'SUP009', 'Tune Hotels', '03-2082-5600',
'hello@tunehotels.com' );
INSERT INTO supplier VALUES ( 'SUP010', 'Banyan Tree', '103-2113-1888',
'kl@banyantree.com' );
```

```
INSERT INTO supplier VALUES ( 'SUP011', 'Aeroline', '03-6258-8800',
'enquiry@aeroline.com.my' );
INSERT INTO supplier VALUES ( 'SUP012', 'Plusliner', '03-2072-8888',
'customercare@plusliner.com' );
INSERT INTO supplier VALUES ( 'SUP013', 'Transnasional', '1-300-88-8582',
'info@transnasional.com' );
INSERT INTO supplier VALUES ( 'SUP014', 'KLIA Ekspres', '03-2267-8000',
'customerenquiry@kliaekspres.com' );
INSERT INTO supplier VALUES ( 'SUP015', 'RapidKL Bus Services', '03-7885-
2585', 'suggest@rapidkl.com.my' );
```

REM *****insert data into the TOUR_GUIDE table

Prompt ***** Populating TOUR_GUIDE table

```
INSERT INTO tour_guide VALUES ( 'GDE001', 'Ahmad', 'Faizal', '019-1112222',
'English, Malay' );
INSERT INTO tour_guide VALUES ( 'GDE002', 'Sarah', 'Lim', '012-3334444',
'English, Mandarin' );
INSERT INTO tour_guide VALUES ( 'GDE003', 'Muthu', 'Kumar', '017-5556666',
'English, Tamil' );
INSERT INTO tour_guide VALUES ( 'GDE004', 'Diana', 'Rose', '016-7778888',
'English, French' );
INSERT INTO tour_guide VALUES ( 'GDE005', 'Kenji', 'Sato', '014-9990000',
'English, Japanese' );
```

REM *****insert data into the AIRLINE table

Prompt ***** Populating AIRLINE table

```
INSERT INTO airline VALUES ( 'SUP001', 'Malaysia Airlines', 'MH' );
INSERT INTO airline VALUES ( 'SUP002', 'AirAsia', 'AK' );
INSERT INTO airline VALUES ( 'SUP003', 'Batik Air', 'OD' );
INSERT INTO airline VALUES ( 'SUP004', 'Firefly', 'FY' );
INSERT INTO airline VALUES ( 'SUP005', 'Singapore Airlines', 'SQ' );
```

REM *****insert data into the HOTEL_PROVIDER table

Prompt ***** Populating HOTEL_PROVIDER table

```
INSERT INTO hotel_provider VALUES ( 'SUP006', 'Shangri-La Kuala Lumpur', 5.0 );
INSERT INTO hotel_provider VALUES ( 'SUP007', 'Hilton Petaling Jaya', 4.5 );
INSERT INTO hotel_provider VALUES ( 'SUP008', 'JW Marriott KL', 5.0 );
INSERT INTO hotel_provider VALUES ( 'SUP009', 'Tune Hotel KLIA2', 3.0 );
INSERT INTO hotel_provider VALUES ( 'SUP010', 'Banyan Tree KL', 5.0 );
```

REM *****insert data into the TRANSPORT_PROVIDER table

Prompt ***** Populating TRANSPORT_PROVIDER table

```
INSERT INTO transport_provider VALUES ( 'SUP011', 'Aeroline');
INSERT INTO transport_provider VALUES ( 'SUP012', 'Plusliner');
INSERT INTO transport_provider VALUES ( 'SUP013', 'Transnasional');
INSERT INTO transport_provider VALUES ( 'SUP014', 'KLIA Ekspres');
INSERT INTO transport_provider VALUES ( 'SUP015', 'RapidKL');
```

REM *****insert data into the SERVICE table

Prompt ***** Populating SERVICE table

```
INSERT INTO service VALUES ( 'SRV001', 'SUP001', 'Flight KUL-BKI', 400.00 );
INSERT INTO service VALUES ( 'SRV002', 'SUP002', 'Flight KUL-LGK', 150.00 );
INSERT INTO service VALUES ( 'SRV003', 'SUP003', 'Flight KUL-PEN', 120.00 );
INSERT INTO service VALUES ( 'SRV004', 'SUP004', 'Flight SZB-KBR', 180.00 );
INSERT INTO service VALUES ( 'SRV005', 'SUP005', 'Flight KUL-SIN', 500.00 );
INSERT INTO service VALUES ( 'SRV006', 'SUP006', 'Deluxe Stay', 550.00 );
INSERT INTO service VALUES ( 'SRV007', 'SUP007', 'Exec Stay', 650.00 );
INSERT INTO service VALUES ( 'SRV008', 'SUP008', 'Standard Stay', 480.00 );
INSERT INTO service VALUES ( 'SRV009', 'SUP009', 'Budget Stay', 150.00 );
INSERT INTO service VALUES ( 'SRV010', 'SUP010', 'Suite Stay', 1200.00 );
INSERT INTO service VALUES ( 'SRV011', 'SUP011', 'Coach Transfer KL', 80.00 );
INSERT INTO service VALUES ( 'SRV012', 'SUP012', 'Bus Transfer Penang', 40.00 );
INSERT INTO service VALUES ( 'SRV013', 'SUP013', 'Express Bus JB', 45.00 );
INSERT INTO service VALUES ( 'SRV014', 'SUP014', 'Airport Train', 55.00 );
INSERT INTO service VALUES ( 'SRV015', 'SUP015', 'City Bus Pass', 10.00 );
```

REM *****insert data into the BOOKING table

Prompt ***** Populating BOOKING table

```
INSERT INTO booking VALUES ( 'BKG001', 'CUST001', 'PKG001', TO_DATE('10-MAY-2025', 'dd-MON-yyyy'), 'Confirmed', 1500.00 );
INSERT INTO booking VALUES ( 'BKG002', 'CUST002', 'PKG002', TO_DATE('12-MAY-2025', 'dd-MON-yyyy'), 'Confirmed', 850.00 );
INSERT INTO booking VALUES ( 'BKG003', 'CUST003', 'PKG003', TO_DATE('15-MAY-2025', 'dd-MON-yyyy'), 'Pending', 450.00 );
INSERT INTO booking VALUES ( 'BKG004', 'CUST004', 'PKG004', TO_DATE('18-MAY-2025', 'dd-MON-yyyy'), 'Confirmed', 1200.00 );
INSERT INTO booking VALUES ( 'BKG005', 'CUST005', 'PKG005', TO_DATE('20-MAY-2025', 'dd-MON-yyyy'), 'Cancelled', 600.00 );
```

REM *****insert data into the ITINERARY table

Prompt ***** Populating ITINERARY table

```
INSERT INTO itinerary VALUES ( 'ITN001', 'PKG001', 1, 'Arrival and Base Camp
Briefing', 'Kinabalu Park HQ', '14:00' );
INSERT INTO itinerary VALUES ( 'ITN002', 'PKG001', 2, 'Ascent to Panalaban',
'Mount Kinabalu Trail', '08:00' );
INSERT INTO itinerary VALUES ( 'ITN003', 'PKG002', 1, 'Island Hopping', 'Langkawi
Archipelago', '09:00' );
INSERT INTO itinerary VALUES ( 'ITN004', 'PKG003', 1, 'Street Art Walk',
'Georgetown, Penang', '10:30' );
INSERT INTO itinerary VALUES ( 'ITN005', 'PKG005', 2, 'Petronas Twin Towers
Visit', 'KLCC', '11:00' );
```

REM *****insert data into the FLIGHT table

Prompt ***** Populating FLIGHT table

```
INSERT INTO flight VALUES ( 'SRV001', 'MH2604', 'KUL', 'BKI', TIMESTAMP '2025-
06-01 08:00:00', TIMESTAMP '2025-06-01 10:30:00', '2h 30m', 'G1' );
INSERT INTO flight VALUES ( 'SRV002', 'AK6112', 'KUL', 'LGK', TIMESTAMP
'2025-06-02 09:15:00', TIMESTAMP '2025-06-02 10:15:00', '1h 00m', 'H4' );
INSERT INTO flight VALUES ( 'SRV003', 'OD2103', 'KUL', 'PEN', TIMESTAMP
'2025-06-03 14:00:00', TIMESTAMP '2025-06-03 15:00:00', '1h 00m', 'K2' );
INSERT INTO flight VALUES ( 'SRV004', 'FY1002', 'SZB', 'KBR', TIMESTAMP
'2025-06-04 11:30:00', TIMESTAMP '2025-06-04 12:45:00', '1h 15m', 'A1' );
INSERT INTO flight VALUES ( 'SRV005', 'SQ107', 'KUL', 'SIN', TIMESTAMP '2025-
06-05 18:45:00', TIMESTAMP '2025-06-05 19:45:00', '1h 00m', 'C3' );
```

REM *****insert data into the HOTEL table

Prompt ***** Populating HOTEL table

```
INSERT INTO hotel VALUES ( 'SRV006', 'Shangri-La KL', 'Kuala Lumpur', 'Deluxe
Room', 550.00 );
INSERT INTO hotel VALUES ( 'SRV007', 'Hilton PJ', 'Petaling Jaya', 'Executive
Suite', 650.00 );
INSERT INTO hotel VALUES ( 'SRV008', 'JW Marriott', 'Bukit Bintang', 'Standard
Room', 480.00 );
INSERT INTO hotel VALUES ( 'SRV009', 'Tune Hotel', 'KLIA2', 'Twin Room', 150.00
);
INSERT INTO hotel VALUES ( 'SRV010', 'Banyan Tree', 'KLCC', 'Signature Suite',
1200.00 );
```

REM *****insert data into the TRANSPORT table

Prompt ***** Populating TRANSPORT table

```
INSERT INTO transport VALUES ( 'SRV011', 'Luxury Coach', 30, 'Corus Hotel KL' );
INSERT INTO transport VALUES ( 'SRV012', 'Express Bus', 40, 'TBS KL' );
INSERT INTO transport VALUES ( 'SRV013', 'Express Bus', 44, 'Penang Sentral' );
INSERT INTO transport VALUES ( 'SRV014', 'Train', 200, 'KL Sentral' );
INSERT INTO transport VALUES ( 'SRV015', 'City Bus', 60, 'Pasar Seni Bus Hub' );
```

REM *****insert data into the PACKAGE_SERVICE table

Prompt ***** Populating PACKAGE_SERVICE table

```
INSERT INTO package_service VALUES ('PKG001', 'SRV001', 350.00, 1 );
INSERT INTO package_service VALUES ('PKG001', 'SRV006', 500.00, 2 );
INSERT INTO package_service VALUES ('PKG002', 'SRV002', 120.00, 1 );
INSERT INTO package_service VALUES ('PKG002', 'SRV009', 130.00, 2 );
INSERT INTO package_service VALUES ('PKG003', 'SRV012', 35.00, 1 );
```

REM *****insert data into the PACKAGE_GUIDE table

Prompt ***** Populating PACKAGE_GUIDE table

```
INSERT INTO package_guide VALUES ( 'PKG001', 'GDE001', 'Lead Guide',
TO_DATE('01-JUN-2025', 'dd-MON-yyyy'), TO_DATE('04-JUN-2025', 'dd-MON-
yyyy') );
INSERT INTO package_guide VALUES ( 'PKG002', 'GDE002', 'Local Escort',
TO_DATE('02-JUN-2025', 'dd-MON-yyyy'), TO_DATE('04-JUN-2025', 'dd-MON-
yyyy') );
INSERT INTO package_guide VALUES ( 'PKG003', 'GDE003', 'Heritage Specialist',
TO_DATE('03-JUN-2025', 'dd-MON-yyyy'), TO_DATE('04-JUN-2025', 'dd-MON-
yyyy') );
INSERT INTO package_guide VALUES ( 'PKG004', 'GDE004', 'Nature Guide',
TO_DATE('04-JUN-2025', 'dd-MON-yyyy'), TO_DATE('08-JUN-2025', 'dd-MON-
yyyy') );
INSERT INTO package_guide VALUES ( 'PKG005', 'GDE005', 'City Translator',
TO_DATE('05-JUN-2025', 'dd-MON-yyyy'), TO_DATE('07-JUN-2025', 'dd-MON-
yyyy') );
```

REM *****insert data into the PAYMENT table

Prompt ***** Populating PAYMENT table

```
INSERT INTO payment VALUES ( 'PAY001', 'BKG001', 1500.00, 'Credit Card',
TIMESTAMP '2025-05-10 10:30:00', 'Success' );
INSERT INTO payment VALUES ( 'PAY002', 'BKG002', 850.00, 'Online Banking',
TIMESTAMP '2025-05-12 14:15:00', 'Success' );
INSERT INTO payment VALUES ( 'PAY003', 'BKG003', 450.00, 'E-Wallet',
TIMESTAMP '2025-05-15 09:00:00', 'Pending' );
```

```
INSERT INTO payment VALUES ( 'PAY004', 'BKG004', 1200.00, 'Credit Card',
TIMESTAMP '2025-05-18 16:45:00', 'Success' );
INSERT INTO payment VALUES ( 'PAY005', 'BKG005', 600.00, 'Online Banking',
TIMESTAMP '2025-05-20 11:20:00', 'Refunded' );
```

REM *****insert data into the RESERVATION table

Prompt ***** Populating RESERVATION table

```
INSERT INTO reservation VALUES ( 'RES001', 'BKG001', TO_DATE('10-MAY-
2025', 'dd-MON-yyyy'), 'Confirmed' );
INSERT INTO reservation VALUES ( 'RES002', 'BKG002', TO_DATE('12-MAY-
2025', 'dd-MON-yyyy'), 'Confirmed' );
INSERT INTO reservation VALUES ( 'RES003', 'BKG003', TO_DATE('15-MAY-
2025', 'dd-MON-yyyy'), 'Pending' );
INSERT INTO reservation VALUES ( 'RES004', 'BKG004', TO_DATE('18-MAY-
2025', 'dd-MON-yyyy'), 'Confirmed' );
INSERT INTO reservation VALUES ( 'RES005', 'BKG005', TO_DATE('20-MAY-
2025', 'dd-MON-yyyy'), 'Cancelled' );
INSERT INTO reservation VALUES ( 'RES006', 'BKG001', TO_DATE('10-MAY-
2025', 'dd-MON-yyyy'), 'Confirmed' );
INSERT INTO reservation VALUES ( 'RES007', 'BKG002', TO_DATE('12-MAY-
2025', 'dd-MON-yyyy'), 'Confirmed' );
INSERT INTO reservation VALUES ( 'RES008', 'BKG003', TO_DATE('15-MAY-
2025', 'dd-MON-yyyy'), 'Pending' );
INSERT INTO reservation VALUES ( 'RES009', 'BKG004', TO_DATE('18-MAY-
2025', 'dd-MON-yyyy'), 'Confirmed' );
INSERT INTO reservation VALUES ( 'RES010', 'BKG005', TO_DATE('20-MAY-
2025', 'dd-MON-yyyy'), 'Cancelled' );
INSERT INTO reservation VALUES ( 'RES011', 'BKG001', TO_DATE('10-MAY-
2025', 'dd-MON-yyyy'), 'Confirmed' );
INSERT INTO reservation VALUES ( 'RES012', 'BKG002', TO_DATE('12-MAY-
2025', 'dd-MON-yyyy'), 'Confirmed' );
INSERT INTO reservation VALUES ( 'RES013', 'BKG003', TO_DATE('15-MAY-
2025', 'dd-MON-yyyy'), 'Pending' );
INSERT INTO reservation VALUES ( 'RES014', 'BKG004', TO_DATE('18-MAY-
2025', 'dd-MON-yyyy'), 'Confirmed' );
INSERT INTO reservation VALUES ( 'RES015', 'BKG005', TO_DATE('20-MAY-
2025', 'dd-MON-yyyy'), 'Cancelled' );
```

REM *****insert data into the FLIGHT_RESERVATION table

Prompt ***** Populating FLIGHT_RESERVATION table

```
INSERT INTO flight_reservation VALUES ( 'RES001', 'SRV001', 'Economy');
INSERT INTO flight_reservation VALUES ( 'RES002', 'SRV002', 'Economy');
INSERT INTO flight_reservation VALUES ( 'RES003', 'SRV003', 'Business');
INSERT INTO flight_reservation VALUES ( 'RES004', 'SRV004', 'Economy');
```

```
INSERT INTO flight_reservation VALUES ( 'RES005', 'SRV005', 'First Class');
```

REM *****insert data into the TICKET table

Prompt ***** Populating TICKET table

```
INSERT INTO ticket VALUES ('TCK001', 'RES001', 'ETKT-990122', '12A',
TO_DATE('11-MAY-2025', 'DD-MON-YYYY'), 'Issued');
INSERT INTO ticket VALUES ('TCK002', 'RES002', 'ETKT-990123', '14C',
TO_DATE('13-MAY-2025', 'DD-MON-YYYY'), 'Issued');
INSERT INTO ticket VALUES ('TCK003', 'RES003', 'ETKT-990124', '2B',
TO_DATE('16-MAY-2025', 'DD-MON-YYYY'), 'Issued');
INSERT INTO ticket VALUES ('TCK004', 'RES004', 'ETKT-990125', '8F',
TO_DATE('19-MAY-2025', 'DD-MON-YYYY'), 'Issued');
INSERT INTO ticket VALUES ('TCK005', 'RES005', 'ETKT-990126', '1A',
TO_DATE('21-MAY-2025', 'DD-MON-YYYY'), 'Cancelled');
```

REM *****insert data into the HOTEL_RESERVATION table

Prompt ***** Populating HOTEL_RESERVATION table

```
INSERT INTO hotel_reservation VALUES ( 'RES006', 'SRV006', TO_DATE('01-JUN-
2025', 'dd-MON-yyyy'), TO_DATE('04-JUN-2025', 'dd-MON-yyyy'), 1 );
INSERT INTO hotel_reservation VALUES ( 'RES007', 'SRV007', TO_DATE('02-JUN-
2025', 'dd-MON-yyyy'), TO_DATE('04-JUN-2025', 'dd-MON-yyyy'), 2 );
INSERT INTO hotel_reservation VALUES ( 'RES008', 'SRV008', TO_DATE('03-JUN-
2025', 'dd-MON-yyyy'), TO_DATE('04-JUN-2025', 'dd-MON-yyyy'), 1 );
INSERT INTO hotel_reservation VALUES ( 'RES009', 'SRV009', TO_DATE('04-JUN-
2025', 'dd-MON-yyyy'), TO_DATE('08-JUN-2025', 'dd-MON-yyyy'), 3 );
INSERT INTO hotel_reservation VALUES ( 'RES010', 'SRV010', TO_DATE('05-JUN-
2025', 'dd-MON-yyyy'), TO_DATE('07-JUN-2025', 'dd-MON-yyyy'), 1 );
```

REM *****insert data into the TRANSPORT_RESERVATION table

Prompt ***** Populating TRANSPORT_RESERVATION table

```
INSERT INTO transport_reservation VALUES ( 'RES011', 'SRV011', 'KL Sentral',
'Mount Kinabalu Base', TIMESTAMP '2025-06-01 08:00:00', TIMESTAMP '2025-06-
01 12:00:00' );
INSERT INTO transport_reservation VALUES ( 'RES012', 'SRV012', 'Penang
Airport', 'Langkawi Jetty', TIMESTAMP '2025-06-02 09:00:00', TIMESTAMP '2025-
06-02 11:00:00' );
INSERT INTO transport_reservation VALUES ( 'RES013', 'SRV013', 'Hotel Lobby',
'Heritage Zone', TIMESTAMP '2025-06-03 10:00:00', TIMESTAMP '2025-06-03
18:00:00' );
```



```
INSERT INTO transport_reservation VALUES ( 'RES014', 'SRV014', 'KLIA', 'Taman  
Negara', TIMESTAMP '2025-06-04 14:00:00', TIMESTAMP '2025-06-04 17:00:00' );  
INSERT INTO transport_reservation VALUES ( 'RES015', 'SRV015', 'Bukit Bintang',  
'Batu Caves', TIMESTAMP '2025-06-05 08:30:00', TIMESTAMP '2025-06-05  
13:00:00' );
```

```
COMMIT;
```

7.1 SQL Queries

Query 1: Retrieve Confirmed Customer Bookings Ordered by Package Price

```
SELECT c.customer_id, c.first_name, c.last_name,
b.booking_id, b.booking_date, tp.package_type, tp.package_price
FROM customer c JOIN booking b ON c.customer_id = b.customer_id
JOIN tour_package tp ON b.package_id = tp.package_id
WHERE b.booking_status = 'Confirmed'
ORDER BY tp.package_price DESC;
```

Operations Performed:

This query utilizes an Inner join across three tables customer, booking and tour_package to consolidate related records. It employs a Filtering operation (WHERE) to isolate specific record states and a Sorting operation (ORDER BY) to organize the output.

How to Use This Command:

A user should execute this command when they need a high-priority list of verified revenue. By filtering for 'Confirmed' status, it removes "pending" or "cancelled" noise from the data. The DESC keyword ensures the report is "top-heavy", allowing management to focus on high-ticket sales first. This is an essential tool for sales audits and identifying VIP customers.

Query 2: Get customers with Successful Payments

```
SELECT c.customer_id, c.first_name, c.last_name
FROM customer c
WHERE c.customer_id IN (
SELECT b.customer_id
FROM booking b
JOIN payment p ON b.booking_id = p.booking_id
WHERE p.payment_status = 'Success');
```

Operations Performed:

This query utilizes a Subquery (Nested Query) combined with an Inner Join to cross-reference multiple data layers. The inner operation joins the booking and payment tables to isolate specific transaction outcomes. It then employs the IN operator as a membership filter, which allows the outer query to pull primary customer details only if their unique ID exists within the filtered set of successful payments. This approach effectively segments the customer base without needing to display transaction-specific details in the final output.

How to Use This Command:

Users should execute this command when they need a "clean" list of verified, paying customers for marketing or loyalty programs. Unlike a standard Join which might procedure duplicate rows for a customer who has made multiple payments, this IN clause ensures each customer is listed only once, regardless of how many bookings they have paid for. It is an ideal tool for generating mailing lists, identifying active users for financial audits or calculating the conversion rate of the total database into paying clientele.

7.2 PL/SQL Stored Procedures

Procedure 1: Add New Payment with Validation

```

CREATE OR REPLACE PROCEDURE add_payment (
  p_payment_id IN payment.payment_id%TYPE,
  p_booking_id IN payment.booking_id%TYPE,
  p_amount IN payment.payment_amount%TYPE,
  p_method IN payment.payment_method%TYPE,
  p_status IN payment.payment_status%TYPE
)
IS
  v_count NUMBER;
BEGIN
  -- Validation: Check if booking exists
  SELECT COUNT(*)
  INTO v_count
  FROM booking
  WHERE booking_id = p_booking_id;

  IF v_count = 0 THEN
    DBMS_OUTPUT.PUT_LINE('Error: Booking does not exist.');
```

--Validate payment method

```

  ELSIF p_method NOT IN ('Credit Card', 'E-Wallet', 'Online Banking') THEN
    DBMS_OUTPUT.PUT_LINE('Error: Invalid payment method.');
```

-- Validate payment status

```

  ELSIF p_status NOT IN ('Success','Pending','Refunded') THEN
    DBMS_OUTPUT.PUT_LINE('Error: Invalid payment status.');
```

ELSE

```

    INSERT INTO payment
    VALUES (p_payment_id, p_booking_id, p_amount, p_method,
    SYSTIMESTAMP, p_status);

    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Payment ' || p_payment_id || ' added
    successfully.');
```

END IF;

```

EXCEPTION
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('Unexpected error occurred.');
```

END;

/

Operations Performed:

This procedure executes a Conditional Insert operation wrapped in a Validation Logic framework. It begins by performing a Lookup/Verification (Select COUNT (*)) to ensure referential integrity before the transaction occurs. The core of the procedure uses a

Multi-branch Condition structure (IF-ELSIF-ELSE) to enforce business rules on specific data fields. Once the data passes these logical gates, the procedure performs a DML Insert, followed by a Transaction Control operation (COMMIT) to persist the changes. Finally, it includes an Exception Handling block (WHEN OTHERS) to catch and report runtime errors without crashing the session.

How to Use This Command:

A user should call this procedure using an EXEC or BEGIN/END block when a customer submits a payment. Instead of raw data entry, this procedure acts as a Security and Integrity Gatekeeper. It is specifically designed to prevent "orphaned" records (payments for bookings that don't exist) and to restrict the payment_method and payment_status fields to a pre-approved list of values. This ensures that the financial data remains standardized, which is critical for accurate accounting and automated financial reporting.

Testing out the procedure:

```
SET SERVEROUTPUT ON;
```

Case 1: Successful Payment

```
EXEC add_payment('PAY100', 'BKG001', 1000, 'Credit Card', 'Success');
```

Case 2: Invalid Booking ID

```
EXEC add_payment('PAY101', 'BKG999', 500, 'Credit Card', 'Success');
```

Case 3: Invalid Payment Status

```
EXEC add_payment('PAY102', 'BKG001', 500, 'Credit Card', 'Done');
```

Case 4: Invalid Payment Method

```
EXEC add_payment('PAY103', 'BKG001', 1000, 'Cash', 'Success');
```

-- Verification: Check the table to see if only PAY100 was added

```
SELECT * FROM payment WHERE payment_id IN('PAY100', 'PAY101', 'PAY102');
```

Procedure 2: Display Booking Total Price

```
CREATE OR REPLACE PROCEDURE get_booking_total_price (
  p_booking_id IN booking.booking_id%TYPE
)
IS
  v_total booking.total_price%TYPE;
BEGIN
  SELECT total_price
  INTO v_total
  FROM booking
  WHERE booking_id = p_booking_id;

  DBMS_OUTPUT.PUT_LINE('Booking ID: ' || p_booking_id);
  DBMS_OUTPUT.PUT_LINE('Total Price: RM' || TO_CHAR(v_total, '999,999.99'));

EXCEPTION
```

```

WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('Error: Booking not found.');
```

WHEN OTHERS THEN

```

    DBMS_OUTPUT.PUT_LINE('Unexpected error occurred.');
```

END;

/

Operations Performed:

This procedure executes a Data Retrieval operation using a Single-Row Select (SELECT INTO) to fetch a specific attribute from the database. It utilizes Input Parameters to localize the search to a specific primary key (booking_id). The procedure also implements Explicit Exception Handling by specifically targeting the NO_DATA_FOUND error, which allows the system to distinguish between a missing record and a general system failure. Additionally, it performs Data Formatting using the TO_CHAR function to convert a numeric value into a localized currency string (RM) for the user interface.

How to Use This Command:

Users or helpdesk staff should call this procedure when they need a quick, read-only view of a customer's financial obligation for a specific trip. By passing the unique booking_id as an argument, the user can instantly verify the cost without writing complex manual queries. It is particularly useful for customer service representatives during live inquiries or for developers to test if the total_price column has been correctly updated by other business logic in the system.

Testing out the procedure:

```
SET SERVEROUTPUT ON
```

Case 1: Valid Booking

```
EXEC get_booking_total_price('BKG001');
```

Case 2: Non-existent Booking

```
EXEC get_booking_total_price('BKG999');
```

7.3 PL/SQL Functions

Function 1: Calculates the total revenue generated from a specific tour package

```

CREATE OR REPLACE FUNCTION get_total_package_revenue (
    p_package_id IN tour_package.package_id%TYPE
)
RETURN NUMBER
IS
    v_total NUMBER(10,2);
BEGIN
    SELECT SUM(p.payment_amount)
    INTO v_total
    FROM payment p
    JOIN booking b ON p.booking_id = b.booking_id
    WHERE b.package_id = p_package_id
    AND p.payment_status = 'Success';
```

```

    RETURN NVL(v_total, 0);
END;
/

```

Operations Performed:

This function executes an Aggregate Calculation using the SUM function to consolidate financial data. It performs an Inner Join between the payment and booking tables to bridge the relations. The logic incorporates a Multi-Condition Filter (WHERE) to ensure that only "Success" statuses contribute to the revenue, preventing pending or failed payments from inflating the results. Finally, it utilizes the NVL (NULL VALUE) function to handle "null" datasets, ensuring a clean numeric output of 0 if no matching records exist.

How to Use This Command:

Users should call this function when they need to perform Product Performance Analysis because this is a stored function and not a procedure, it can be seamlessly integrated directly into other SELECT statements. For instance, to compare the revenue of all packages in a single report. It is a vital tool for the financial and marketing departments to identify which tour packages are the most profitable and to assist in calculating the Return on Investment (ROI) for specific travel offerings.

Testing out the function:

```

-- Test for a specific package (e.g., PKG001)
SELECT package_id, package_type, get_total_package_revenue(package_id) AS
total_revenue
FROM tour_package
WHERE package_id = 'PKG001';

```

```

-- Testing all packages to show a comparative report
SELECT package_id, package_type,
TO_CHAR(get_total_package_revenue(package_id), '$999,990.00') AS
revenue_report
FROM tour_package;

```

Function 2: Categories a customer's spending level based on their total booking payments

```

CREATE OR REPLACE FUNCTION get_customer_spending_level (
    p_customer_id IN customer.customer_id%TYPE
)
RETURN VARCHAR2
IS
    v_total NUMBER;
BEGIN
    SELECT SUM(p.payment_amount)
    INTO v_total
    FROM payment p
    JOIN booking b ON p.booking_id = b.booking_id
    WHERE b.customer_id = p_customer_id

```

```

AND p.payment_status = 'Success';

IF v_total IS NULL OR v_total = 0 THEN
    RETURN 'No Spending';

    ELSIF v_total < 1000 THEN
        RETURN 'Low Spender';

    ELSIF v_total <= 3000 THEN
        RETURN 'Medium Spender';

    ELSE
        RETURN 'High Spender';
    END IF;
END;
/

```

Operations Performed:

This function executes an Aggregate Data Analysis by combining a SUM function with an Inner Join across the payment and booking tables. It uses Relational Filtering to ensure that only "Success" transaction tied to a specific customer_id are calculated. The core logic relies on a Scalar Evaluation. The function takes a raw numeric sum and passes it through a Condition Classification logic (IF-ELSIF-ELSE). This transforms quantitative data (the dollar amount) into quantitative data (the category string).

How to Use This Command:

A user or automated system should call this function to perform Customer Segmentation on the fly because it returns a simple string, it is perfect for use in CRM (Customer Relationship Management) dashboard or targeted marketing queries. For instance, a marketing manager could run a single query using this function to instantly identify all "High Spender" customers to send them an exclusive VIP discount code or identify "Low spenders" who might need a promotional nudge to increase their booking frequency.

Test out the function:

```

SET SERVEROUTPUT ON;
DECLARE
    v_level VARCHAR2(50);
    v_cust_id customer.customer_id%TYPE := 'CUST001';
BEGIN
    v_level := get_customer_spending_level(v_cust_id);
    DBMS_OUTPUT.PUT_LINE('The spending level for ' || v_cust_id || ' is: ' ||
v_level);
END;
/

```